

Exercício: Academia no Boilerplate (Minhas Matrículas)

Turma: LionsDev

Tópicos: Boilerplate, rotas protegidas com JWT, dono do recurso via token, Mongoose com `ObjectId` / `ref`, CRUD por dono, regra de negócio no service, validações e status codes.

1. Contexto

No Módulo 08 você criou a API da Academia Lions com tudo no `server.js`. Agora a mesma ideia vai para o boilerplate em camadas, e cada matrícula passa a pertencer a um usuário logado.

Imagine um app em que cada gerente de unidade faz login e administra somente as matrículas dele. Ninguém vê as matrículas de outra pessoa.

Mesmo padrão do recurso `Livro` da aula: recurso ligado ao dono (`req.usuario.id`).

2. Ponto de Partida: o Boilerplate

Partimos do boilerplate LionsDev: <https://github.com/nicolassmotta/boilerplate-lions-dev.git>

1. Clone o boilerplate e rode `npm install`.
2. Crie o `.env` a partir do `.env.example` (`MONGO_URI`, `JWT_SECRET`, `JWT_EXPIRES_IN`, `BCRYPT_SALT_ROUNDS`).
3. Suba o servidor e confirme `GET /` respondendo.

O boilerplate já vem com a camada de `Usuario`, o cadastro/login com bcrypt e JWT, o middleware `autenticar` (que preenche `req.usuario = { id, email }`), o helper `criarErro` e o middleware central de erro.

Você não mexe na autenticação. Vai criar o recurso `Matricula` e ligá-lo ao dono.

Antes de começar, pegue seu token: faça cadastro/login, copie o `token` e envie em todas as rotas novas no header `Authorization: Bearer SEU_TOKEN`.

3. Regras do Sistema (valem para todas as rotas do recurso)

1. **Toda rota é protegida.** `router.use(authenticar)` no topo do arquivo de rotas.
2. **O dono vem do token** (`req.usuario.id`), nunca do body.
3. **Toda consulta filtra pelo dono.**
4. **Filtre pelo dono já na consulta** (`{ _id: idMatricula, usuario: idDoUsuario }`).
Se não achar → `404` , sem `if` de autorização.
5. **Listagem nunca dá 404:** sem matrículas, devolva array vazio.

4. Arquivos que você vai criar

Camada	Arquivo
Model	<code>src/models/matricula.model.js</code>
Repository	<code>src/repositories/matricula.repository.js</code>
Service	<code>src/services/matricula.service.js</code>
Controller	<code>src/controllers/matricula.controller.js</code>
Routes	<code>src/routes/matricula.routes.js</code>

5. Etapa 1 — Model **Matricula**

Crie `src/models/matricula.model.js` . Campos:

- `nomeAluno` : `String` , obrigatório, `trim` .
- `idade` : `Number` , obrigatório.
- `modalidade` : `String` , obrigatório, `enum`: ["Musculação", "Funcional", "Dança"] .
- `plano` : `String` , obrigatório, `enum`: ["Mensal", "Trimestral", "Semestral"] .
- `dataMatricula` : `String` , obrigatório (ex.: `"2026-06-15"`) .
- `valorMensal` : `Number` (a **API** calcula).
- `valorTotal` : `Number` (a **API** calcula).
- `status` : `String` , `enum`: ["Ativa", "Pausada", "Cancelada"] , `default`: "Ativa" .
- `usuario` : `ObjectId` , `ref`: "Usuario" , obrigatório.
- Ative `timestamps`: `true` .

Critérios de aceite: `nomeAluno` , `idade` , `modalidade` , `plano` , `dataMatricula` e `usuario` são obrigatórios; `modalidade` , `plano` e `status` só aceitam os valores das listas.

Conceito novo: `enum` e o campo de dono (`ObjectId` + `ref`)

- `enum` trava `modalidade` , `plano` e `status` em listas fechadas (valor fora da lista → `400`).
- `usuario` guarda o dono da matrícula: uma referência ao `_id` de um `Usuario` .

```
import mongoose from "mongoose";
// ...dentro do Schema
modalidade: { type: String, required: true, enum: ["Musculação",
"Funcional", "Dança"] },
usuario: { type: mongoose.Schema.Types.ObjectId, ref: "Usuario",
required: true },
```

6. Etapa 2 — Repository

Crie `src/repositories/matricula.repository.js` (use o `usuario.repository.js` do boilerplate como referência do formato):

- `criar(dados)` → `Matricula.create(dados)` .
- `listarPorUsuario(idUsuario)` → `Matricula.find({ usuario: idUsuario }).sort({ createdAt: -1 })` .
- `buscarPorIdDoDono(idMatricula, idUsuario)` → `Matricula.findOne({ _id: idMatricula, usuario: idUsuario })` .
- `atualizarPorIdDoDono(idMatricula, idUsuario, dados)` → `findOneAndUpdate(filtro, dados, { new: true, runValidators: true })` .
- `deletarPorIdDoDono(idMatricula, idUsuario)` → `findOneAndDelete(filtro)` .
- Exporte tudo no objeto `MatriculaRepository` .

Critérios de aceite: toda consulta filtra por `usuario` ; retornam `null` quando não encontram.

7. Etapa 3 — Service (regras de negócio)

Crie `src/services/matricula.service.js` . Regras (iguais às do Módulo 08):

1. **Valor mensal** pela modalidade: **Musculação** = 90 · **Funcional** = 120 · **Dança** = 100.
2. **Valor total** pelo plano:
 - **Mensal** : 1 mensalidade.
 - **Trimestral** : 3 mensalidades com 10% de desconto.
 - **Semestral** : 6 mensalidades com 15% de desconto.

Como calcular **valorMensal** e **valorTotal** no service (use objetos em vez de uma pilha de **if**):

```
const PRECOS = { "Musculação": 90, "Funcional": 120, "Dança": 100 };
const MESES = { "Mensal": 1, "Trimestral": 3, "Semestral": 6 };
const DESCONTO = { "Mensal": 0, "Trimestral": 0.10, "Semestral": 0.15 };

const valorMensal = PRECOS[modalidade];
const valorTotal = valorMensal * MESES[plano] * (1 - DESCONTO[plano]);
```

Ex.: Trimestral de Funcional → $120 * 3 * (1 - 0,10) = 324$.

Funções:

- **registrar(idDoUsuario, dados)** : calcula **valorMensal** e **valorTotal**, monta o objeto com **usuario: idDoUsuario** e chama **MatriculaRepository.criar(...)**.
- **listarMinhas(idDoUsuario)** : retorna a lista do usuário.
- **buscarMinha(idDoUsuario, idMatricula)** : **buscarPorIdDoDono(...)**; se **null** → **criarErro("Matrícula não encontrada.", 404)**.
- **atualizarMinha(idDoUsuario, idMatricula, dados)** : **atualizarPorIdDoDono(...)**; se **null** → **404**.
- **removerMinha(idDoUsuario, idMatricula)** : **deletarPorIdDoDono(...)**; se **null** → **404**; se ok → **{ message: "Matrícula removida com sucesso." }**.

Critérios de aceite: **Mensal** de **Musculação** grava **valorTotal** 90; **Trimestral** de **Funcional** aplica os 10% ($3 \times 120 \times 0,9 = 324$); o dono salvo é sempre o **idDoUsuario** recebido.

8. Etapa 4 — Controller

Crie **src/controllers/matricula.controller.js** (com **try/catch** e **next(error)**):

- **registrar(req, res, next)** → **req.usuario.id** + **req.body**; responde **201**.

- `listarMinhas(req, res, next) → req.usuario.id` ; responde `200` com `{ matriculas }` .
- `buscarMinha(req, res, next) → req.usuario.id + req.params.id` ; responde `200` .
- `atualizarMinha(req, res, next) → req.usuario.id + req.params.id + req.body` ; responde `200` .
- `removerMinha(req, res, next) → req.usuario.id + req.params.id` ; responde `200` .

9. Etapa 5 — Routes e registro no `app.js`

Crie `src/routes/matricula.routes.js` :

- Importe `Router` , o `MatriculaController` e `autenticar` .
- `router.use(autenticar)` no topo.
- `post("/")` , `get("/")` , `get("/:id")` , `patch("/:id")` , `delete("/:id")` apontando para as funções do controller.

No `src/app.js` , antes do middleware 404:

```
import matriculaRoutes from "../routes/matricula.routes.js";
app.use("/api/matriculas", matriculaRoutes);
```

Critérios de aceite: chamar `/api/matriculas/...` sem token → `401` ; a aplicação sobe sem erros.

10. Rotas finais

Método	Rota	Proteção	Descrição
POST	<code>/api/matriculas</code>	JWT	Criar matrícula (dono = você)
GET	<code>/api/matriculas</code>	JWT	Listar minhas matrículas
GET	<code>/api/matriculas/:id</code>	JWT	Ver uma matrícula minha
PATCH	<code>/api/matriculas/:id</code>	JWT	Atualizar uma matrícula minha
DELETE	<code>/api/matriculas/:id</code>	JWT	Remover uma matrícula minha

11. Fluxo de Testes (use Postman)

1. Cadastro/login → copie o **token**.
2. **POST** **Mensal** + **Musculação** → **valorTotal** = 90.
3. **POST** **Trimestral** + **Funcional** → **valorTotal** = 324 (desconto de 10% aplicado).
4. **GET** **/api/matriculas** → só as suas.
5. **PATCH** **/api/matriculas/:id** com **{ "status": "Pausada" }** → **200**.
6. **DELETE** **/api/matriculas/:id** → **200**; repita → **404**.
7. Com um segundo usuário, tente ver/editar/remover uma matrícula do primeiro → **404**.
8. Qualquer rota sem token → **401**.

12. Desafios Bônus

1. **GET** **/api/matriculas/busca?modalidade=func** : filtra as suas matrículas pela modalidade.
2. **GET** **/api/matriculas/resumo** : em JavaScript, conte matrículas por **status** e some o **valorTotal** das suas matrículas **Ativa**.
3. Bloqueie a criação de uma nova matrícula **Ativa** se o mesmo **nomeAluno** já tiver uma matrícula **Ativa** sua.

LionsDev • Professor Nicolas Cardoso Motta

Academia no Boilerplate (Auth + camadas) - Módulo 09